SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
INSTITUTE OF CSE
CSA15- CLOUD COMPUTING AND BIG DATA ANALYTICS

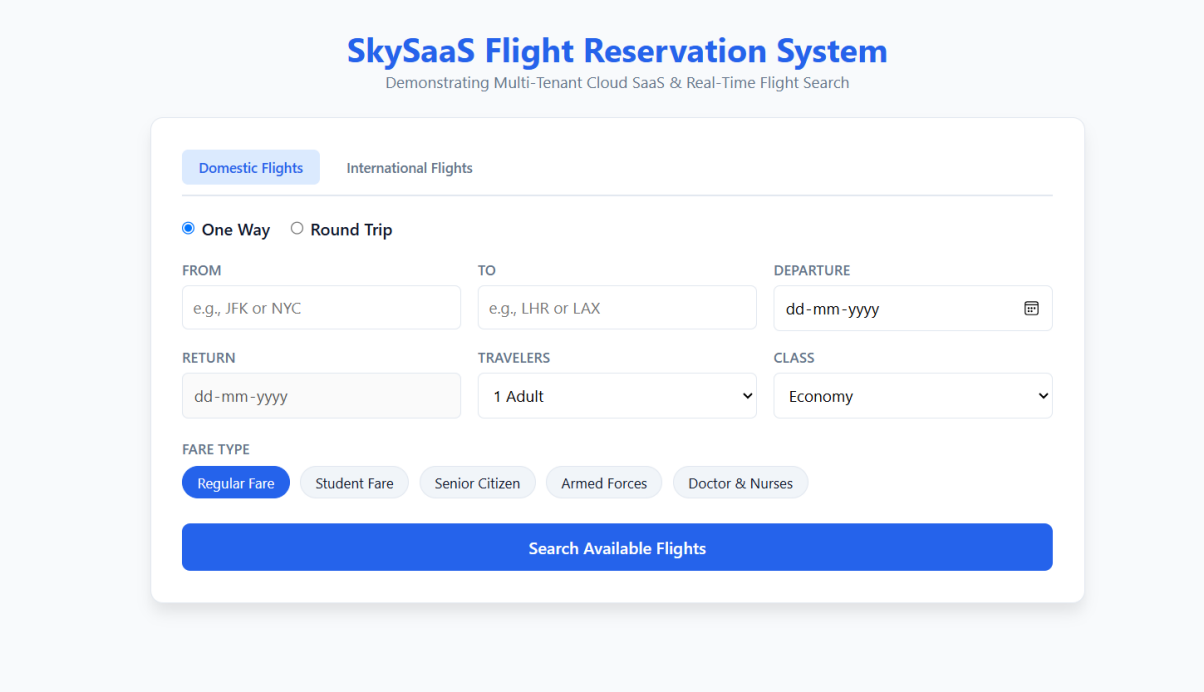
EXPERIMENT 1: Create a simple cloud software application for Flight Reservation System using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as one way, round trip, From, to, Departure, Departure, Travelers & Class, Fare Type, International and Domestic Flights etc.

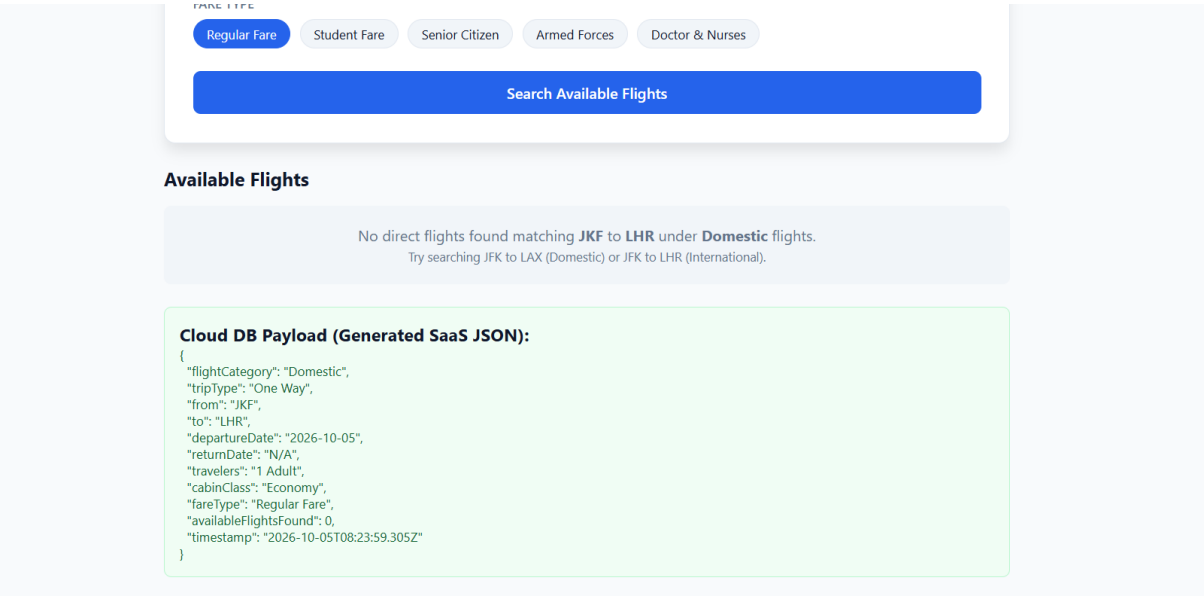
AIM:
To create a simple cloud software application for the Flight Reservation System using a Cloud Service Provider and demonstrate Software as a Service (SaaS).

PROCEDURE:
1. Sign in to the chosen Cloud Service Provider (for example Google Cloud, Zoho Creator or Microsoft Azure) and create a new application project.
2. Design the input form and add the required fields: trip type (one way / round trip), From, To, Departure date, Return date, Travelers, Class, Fare Type and Flight Category (International / Domestic).
3. Create the database table or data store and link each form field to a column.
4. Write the application logic to validate the input, process the data and store the records.
5. Publish the application, open its public URL in a browser, enter sample data and verify the output.

PSEUDOCODE / ALGORITHM:
```
START
  DECLARE tripType, from, to, departDate, returnDate, travelers, class, fareType, category
  INPUT tripType, from, to, departDate, travelers, class, fareType, category
  IF from = "" OR to = "" OR departDate = "" OR travelers < 1 THEN
    PRINT "Required field missing"
    GOTO input form
  END IF
  IF tripType = "Round Trip" THEN
    INPUT returnDate
  ELSE
    returnDate = NULL
  END IF
  flights = SEARCH(from, to, departDate, returnDate, category)
  fare = baseFare * travelers
  IF class = "Business" THEN fare = fare * 1.5
  IF fareType = "Flexible" THEN fare = fare + 500
  bookingID = GENERATE_ID()
  INSERT INTO bookings VALUES (bookingID, tripType, from, to, departDate, returnDate,
travelers, class, fareType, category, fare)
  PRINT "Booking confirmed", bookingID, fare
END
```

OUTPUT:





RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 2: Create a simple cloud software application for Property Buying & Rental process (In Chennai city) using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as Buy, Rent, commercial, Rental Agreement, Rental Agreement, Area, place, Range, property value etc.

AIM:

To create a simple cloud software application for the Property Buying and Rental process in Chennai city using a Cloud Service Provider and demonstrate Software as a Service (SaaS).

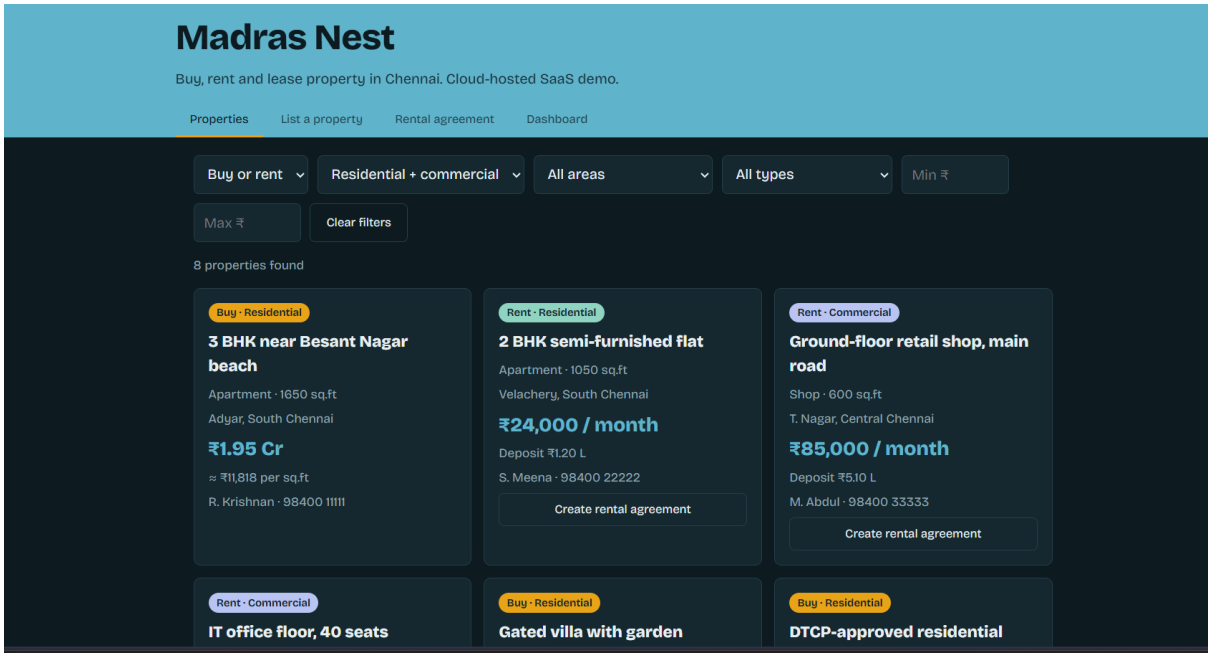
PROCEDURE:

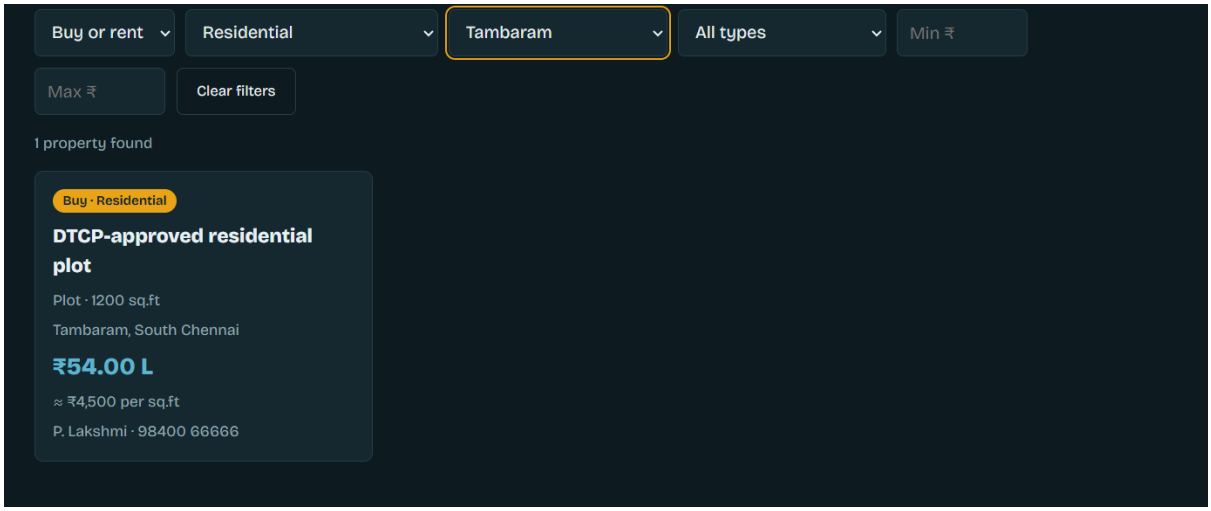
1. Sign in to the chosen Cloud Service Provider (for example Google Cloud, Zoho Creator or Microsoft Azure) and create a new application project.
2. Design the input form and add the required fields: Buy / Rent option, Residential / Commercial type, Rental Agreement, Area, Place, Price Range and Property Value.
3. Create the database table or data store and link each form field to a column.
4. Write the application logic to validate the input, process the data and store the records.
5. Publish the application, open its public URL in a browser, enter sample data and verify the output.

PSEUDOCODE / ALGORITHM:

```
START
  DECLARE option, propType, agreement, area, place, minPrice, maxPrice
  INPUT option, propType, agreement, area, place, minPrice, maxPrice
  IF area = "" OR place = "" OR option = "" THEN
    PRINT "Required field missing"
    GOTO input form
  END IF
  properties = SELECT * FROM property WHERE area = area AND place = place AND type
= propType AND value BETWEEN minPrice AND maxPrice
  FOR EACH p IN properties
    IF option = "Rent" THEN
      PRINT p.name, p.monthlyRent, p.agreementTerms
    ELSE IF option = "Buy" THEN
      PRINT p.name, p.propertyValue
    END IF
  END FOR
  INSERT INTO enquiry VALUES (option, propType, area, place, minPrice, maxPrice)
  PRINT "Search completed"
END
```

OUTPUT:





RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

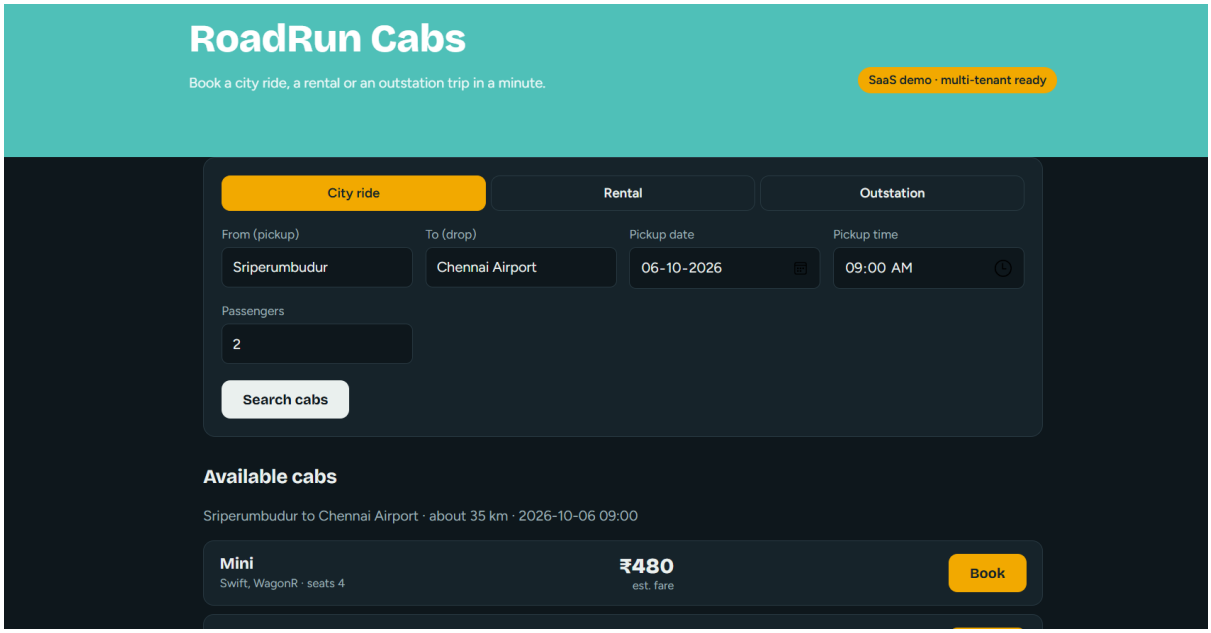
EXPERIMENT 3: Create a simple cloud software application for Car Booking Reservation System using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as search for cabs, from, to, rental, out station, Package type, hours/Days etc.

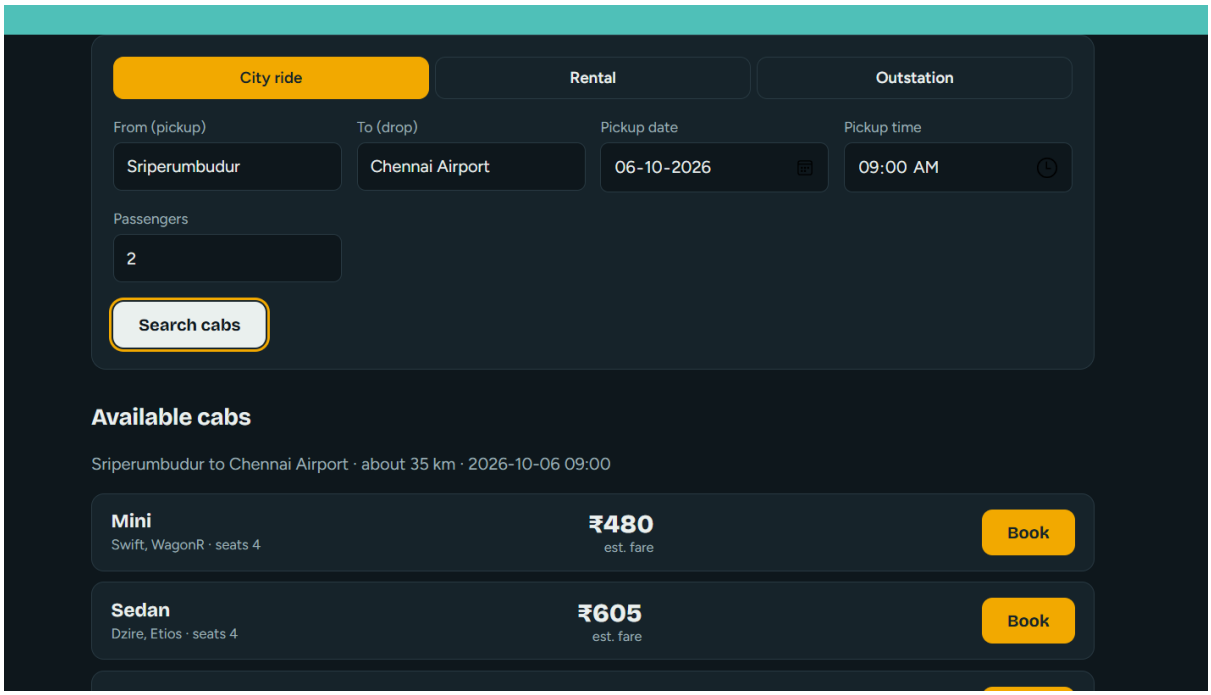
AIM:
To create a simple cloud software application for the Car Booking Reservation System using a Cloud Service Provider and demonstrate Software as a Service (SaaS).

PROCEDURE:
1. Sign in to the chosen Cloud Service Provider (for example Google Cloud, Zoho Creator or Microsoft Azure) and create a new application project.
2. Design the input form and add the required fields: Search for cabs, From, To, Trip Type (Local Rental / Out Station), Package Type and Hours / Days.
3. Create the database table or data store and link each form field to a column.
4. Write the application logic to validate the input, process the data and store the records.
5. Publish the application, open its public URL in a browser, enter sample data and verify the output.

PSEUDOCODE / ALGORITHM:
```
START
  DECLARE from, to, tripType, packageType, hours, days, distance, fare
  INPUT from, to, tripType, packageType, hours, days
  IF from = "" OR to = "" THEN
    PRINT "Required field missing"
    GOTO input form
  END IF
  IF tripType = "Local Rental" THEN
    fare = hours * ratePerHour
  ELSE IF tripType = "Out Station" THEN
    distance = GET_DISTANCE(from, to)
    fare = distance * ratePerKm * days
  END IF
  cabs = SELECT * FROM cabs WHERE available = TRUE AND type = packageType
  bookingID = GENERATE_ID()
  INSERT INTO bookings VALUES (bookingID, from, to, tripType, packageType, fare)
  PRINT "Cab booked", bookingID, fare
END
```

OUTPUT:





RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 4: Create a simple cloud software application for Library book reservation system for SIMATS library using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as Title of the book, author, publications, Year, edition, no of copies, Rack no, status (taken/returned) etc.

AIM:

To create a simple cloud software application for the Library book reservation system of the SIMATS library using a Cloud Service Provider and demonstrate Software as a Service (SaaS).

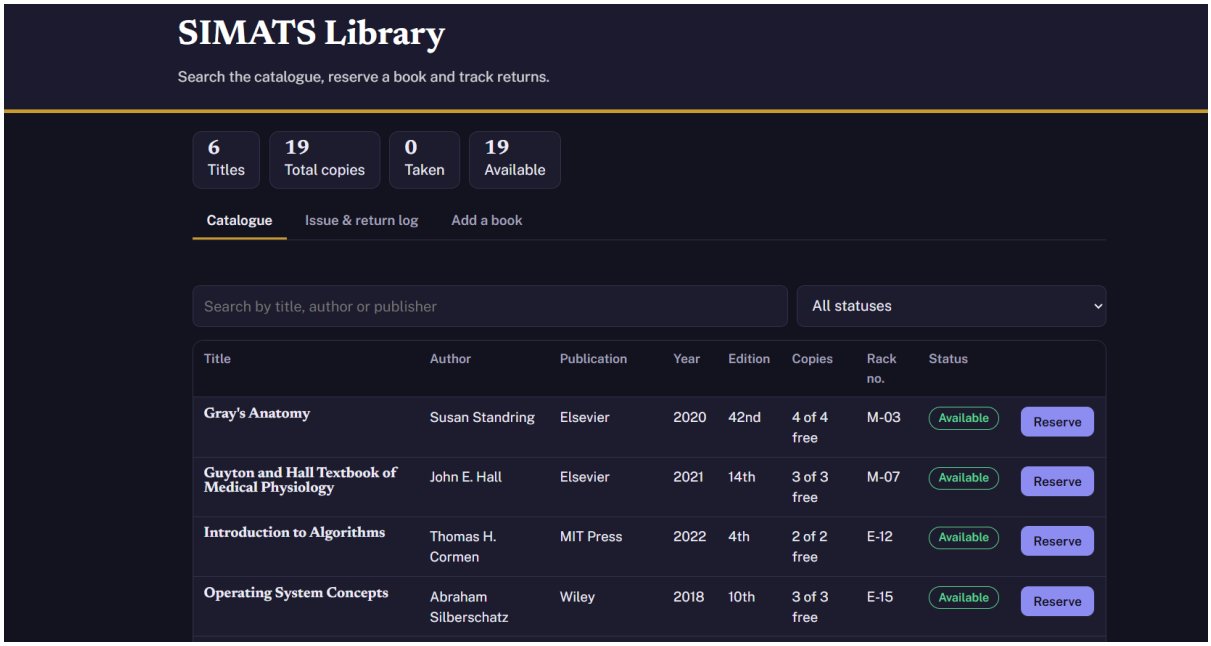
PROCEDURE:

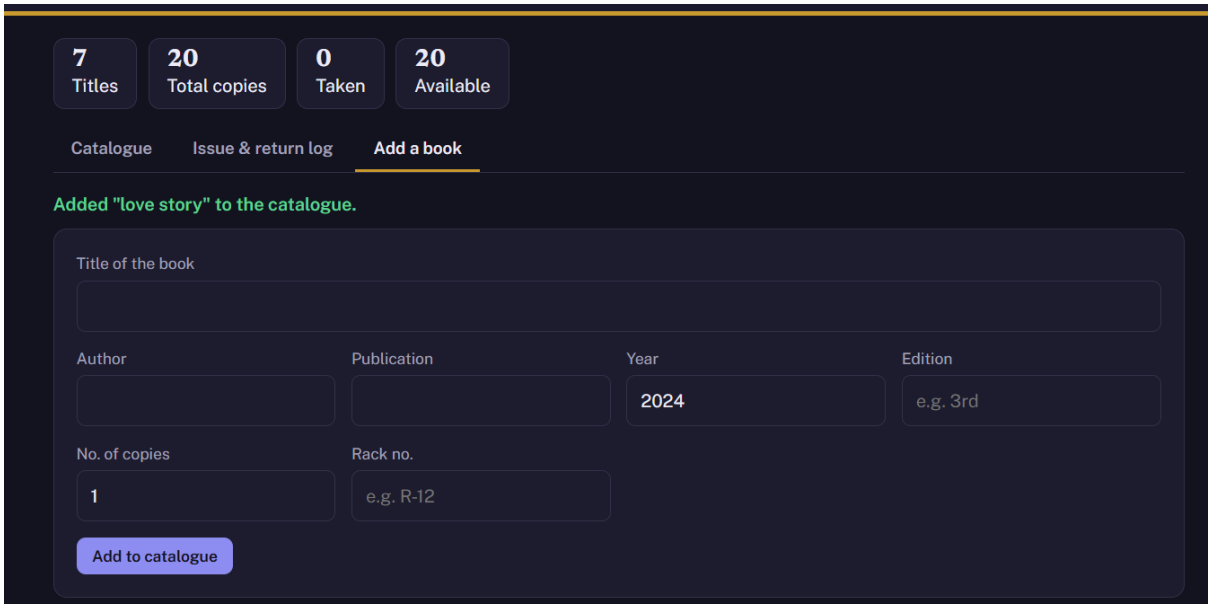
1. Sign in to the chosen Cloud Service Provider (for example Google Cloud, Zoho Creator or Microsoft Azure) and create a new application project.
2. Design the input form and add the required fields: Title of the book, Author, Publications, Year, Edition, Number of Copies, Rack Number and Status (Taken / Returned).
3. Create the database table or data store and link each form field to a column.
4. Write the application logic to validate the input, process the data and store the records.
5. Publish the application, open its public URL in a browser, enter sample data and verify the output.

PSEUDOCODE / ALGORITHM:

```
START
  DECLARE title, author, publication, year, edition, copies, rackNo, status, action
  INPUT title, author, publication, year, edition, copies, rackNo, action
  IF title = "" OR author = "" THEN
    PRINT "Required field missing"
    GOTO input form
  END IF
  IF action = "Reserve" AND copies > 0 THEN
    copies = copies - 1
    status = "Taken"
  ELSE IF action = "Return" THEN
    copies = copies + 1
    status = "Returned"
  END IF
  IF copies = 0 THEN
    PRINT "Not Available"
  END IF
  UPDATE books SET copies = copies, status = status WHERE title = title
  PRINT title, rackNo, copies, status
END
```

OUTPUT:





RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 5: Create a simple cloud software application for Mark Sheet Processing System using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as name of the student, Reg no of the student, Subjects, Marks, Average, percentage, rank, overall performance etc.

AIM:

To create a simple cloud software application for the Mark Sheet Processing System using a Cloud Service Provider and demonstrate Software as a Service (SaaS).

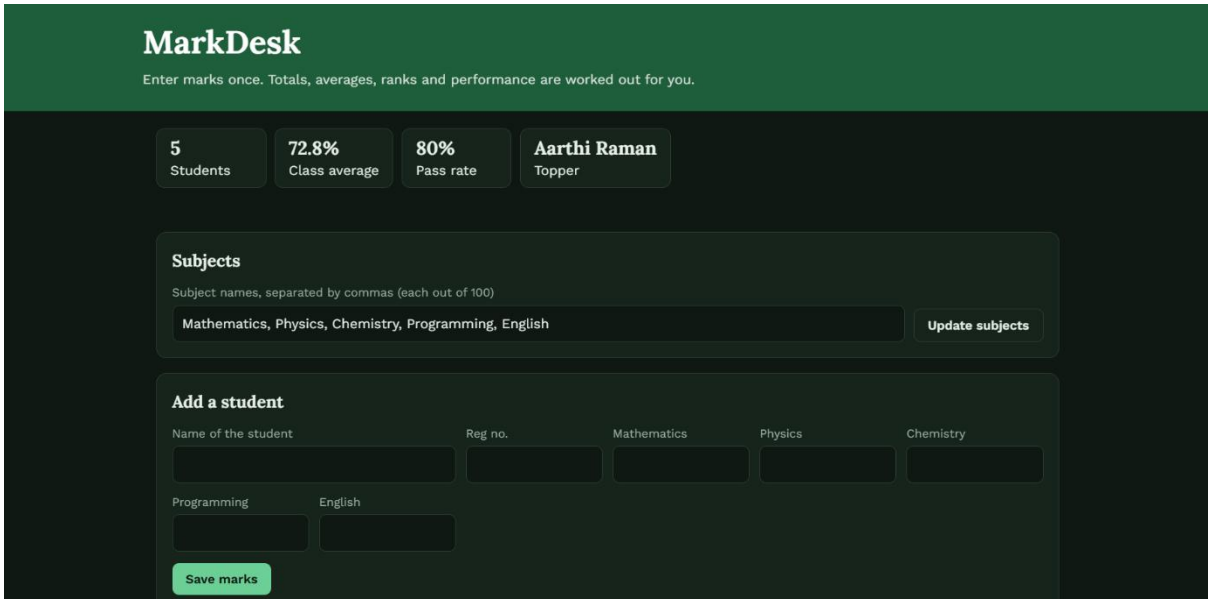
PROCEDURE:

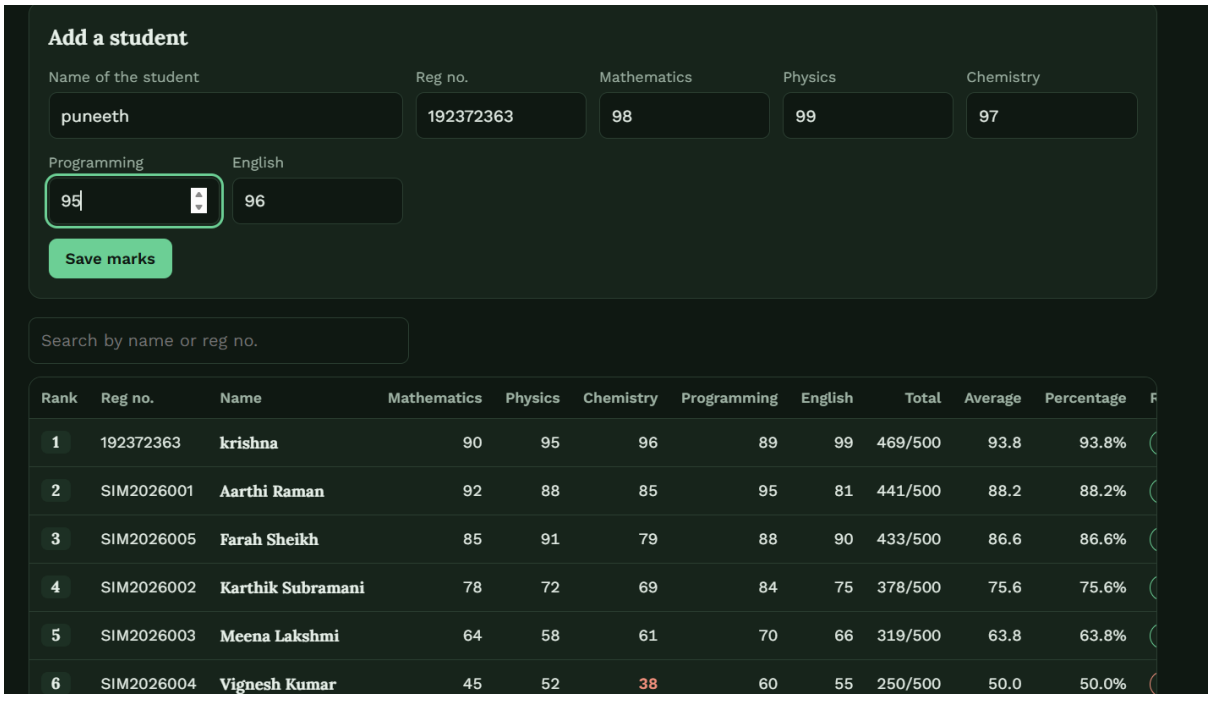
1. Sign in to the chosen Cloud Service Provider (for example Google Cloud, Zoho Creator or Microsoft Azure) and create a new application project.
2. Design the input form and add the required fields: Student Name, Register Number, Subjects, Marks, Average, Percentage, Rank and Overall Performance.
3. Create the database table or data store and link each form field to a column.
4. Write the application logic to validate the input, process the data and store the records.
5. Publish the application, open its public URL in a browser, enter sample data and verify the output.

PSEUDOCODE / ALGORITHM:

```
START
  DECLARE name, regNo, n, marks[], total, average, percentage, rank, performance
  INPUT name, regNo, n
  total = 0
  FOR i = 1 TO n
    INPUT marks[i]
    total = total + marks[i]
  END FOR
  average = total / n
  percentage = total / (n * 100) * 100
  rank = POSITION OF total IN students SORTED BY total DESC
  IF percentage >= 75 THEN performance = "Distinction"
  ELSE IF percentage >= 60 THEN performance = "First Class"
  ELSE IF percentage >= 50 THEN performance = "Pass"
  ELSE performance = "Fail"
  END IF
  INSERT INTO marksheet VALUES (regNo, name, total, average, percentage, rank,
performance)
  PRINT name, regNo, total, average, percentage, rank, performance
END
```

OUTPUT:





Rank	Reg no.	Name	Mathematics	Physics	Chemistry	Programming	English	Total	Average	Percentage
1	192372363	krishna	90	95	96	89	99	469/500	93.8	93.8%
2	SIM2026001	Aarthi Raman	92	88	85	95	81	441/500	88.2	88.2%
3	SIM2026005	Farah Sheikh	85	91	79	88	90	433/500	86.6	86.6%
4	SIM2026002	Karthik Subramani	78	72	69	84	75	378/500	75.6	75.6%
5	SIM2026003	Meena Lakshmi	64	58	61	70	66	319/500	63.8	63.8%
6	SIM2026004	Vignesh Kumar	45	52	38	60	55	250/500	50.0	50.0%

RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 6: Create a simple cloud software application for Payroll Processing System using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as Name of the employee, experience in present org, over all experience, basic pay, HRA, DA, TA, net pay, gross pay etc.

AIM:

To create a simple cloud software application for the Payroll Processing System using a Cloud Service Provider and demonstrate Software as a Service (SaaS).

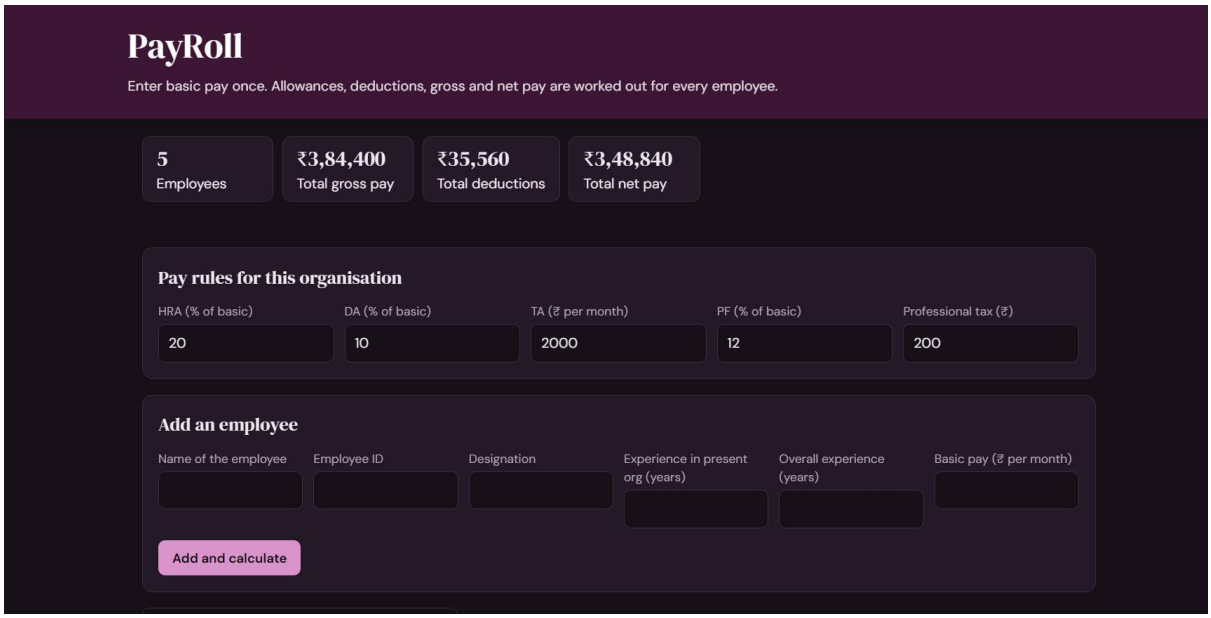
PROCEDURE:

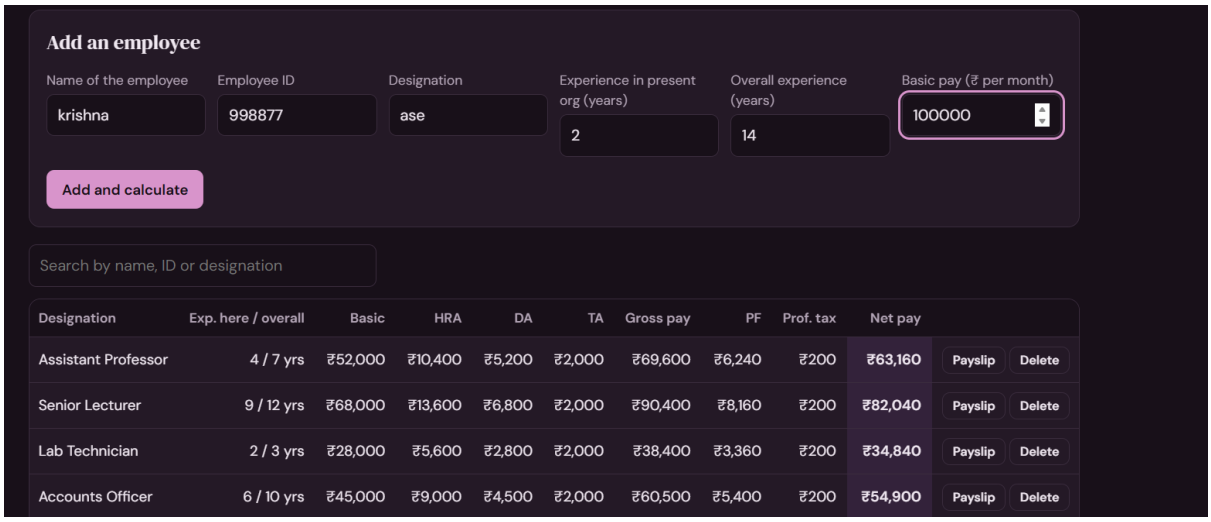
1. Sign in to the chosen Cloud Service Provider (for example Google Cloud, Zoho Creator or Microsoft Azure) and create a new application project.
2. Design the input form and add the required fields: Employee Name, Experience in present organisation, Overall Experience, Basic Pay, HRA, DA, TA, Deductions, Gross Pay and Net Pay.
3. Create the database table or data store and link each form field to a column.
4. Write the application logic to validate the input, process the data and store the records.
5. Publish the application, open its public URL in a browser, enter sample data and verify the output.

PSEUDOCODE / ALGORITHM:

```
START
  DECLARE name, presentExp, totalExp, basic, hraPercent, daPercent, hra, da, ta,
deductions, grossPay, netPay
  INPUT name, presentExp, totalExp, basic, hraPercent, daPercent, ta, deductions
  IF name = "" OR basic <= 0 THEN
    PRINT "Required field missing"
    GOTO input form
  END IF
  hra = basic * hraPercent / 100
  da = basic * daPercent / 100
  grossPay = basic + hra + da + ta
  netPay = grossPay - deductions
  INSERT INTO payroll VALUES (name, basic, hra, da, ta, deductions, grossPay, netPay)
  PRINT "Pay Slip", name, grossPay, netPay
END
```

OUTPUT:





RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 7: Create a simple cloud software application for Students Attendance System using any Cloud Service Provider to demonstrate SaaS include the necessary fields such as name of the students, course, present, Absent, course attendance, OD request, grade, attendance percentage, etc.

AIM:

To create a simple cloud software application for the Students Attendance System using a Cloud Service Provider and demonstrate Software as a Service (SaaS).

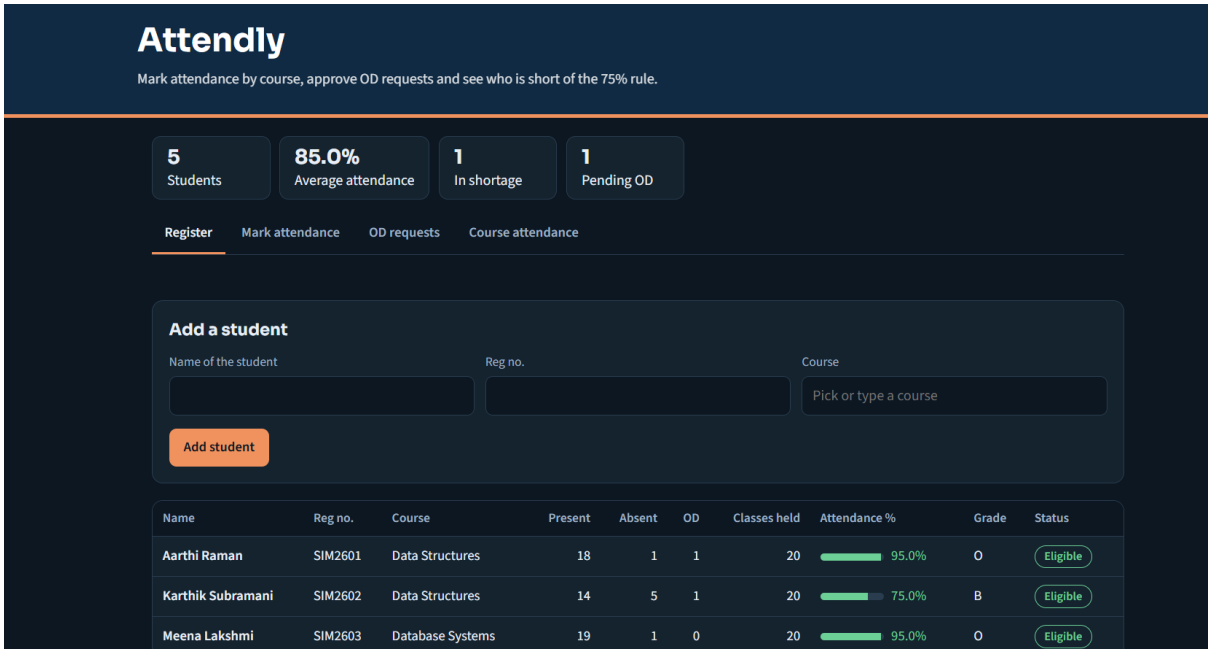
PROCEDURE:

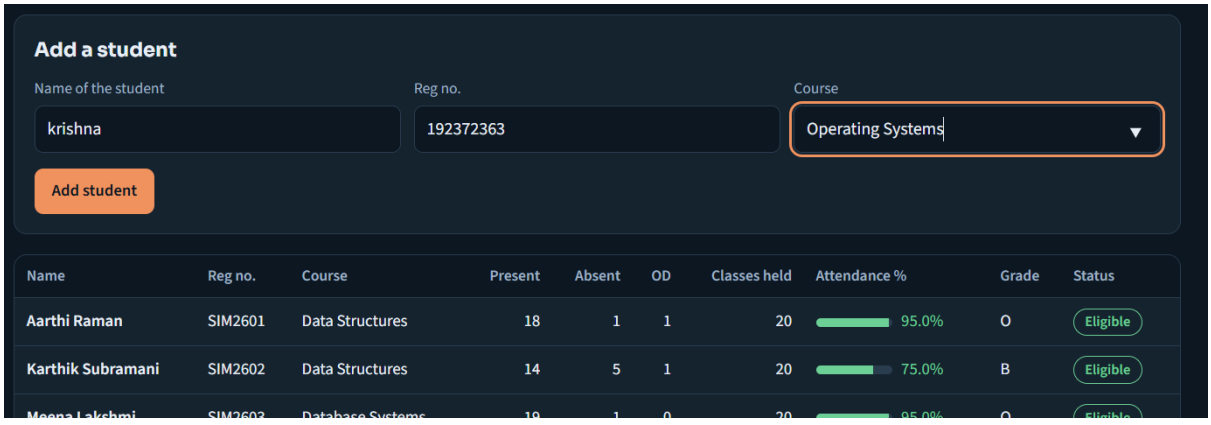
1. Sign in to the chosen Cloud Service Provider (for example Google Cloud, Zoho Creator or Microsoft Azure) and create a new application project.
2. Design the input form and add the required fields: Student Name, Course, Present, Absent, Course Attendance, OD Request, Grade and Attendance Percentage.
3. Create the database table or data store and link each form field to a column.
4. Write the application logic to validate the input, process the data and store the records.
5. Publish the application, open its public URL in a browser, enter sample data and verify the output.

PSEUDOCODE / ALGORITHM:

```
START
  DECLARE name, course, present, absent, odDays, totalClasses, attended, percentage, grade
  INPUT name, course, present, absent, odDays
  totalClasses = present + absent
  attended = present + odDays
  percentage = attended / totalClasses * 100
  IF percentage >= 90 THEN grade = "A"
  ELSE IF percentage >= 80 THEN grade = "B"
  ELSE IF percentage >= 75 THEN grade = "C"
  ELSE grade = "D"
  END IF
  IF percentage < 75 THEN
    PRINT "Shortage of attendance"
  END IF
  INSERT INTO attendance VALUES (name, course, present, absent, odDays, percentage,
grade)
  PRINT name, course, percentage, grade
END
```

OUTPUT:





RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

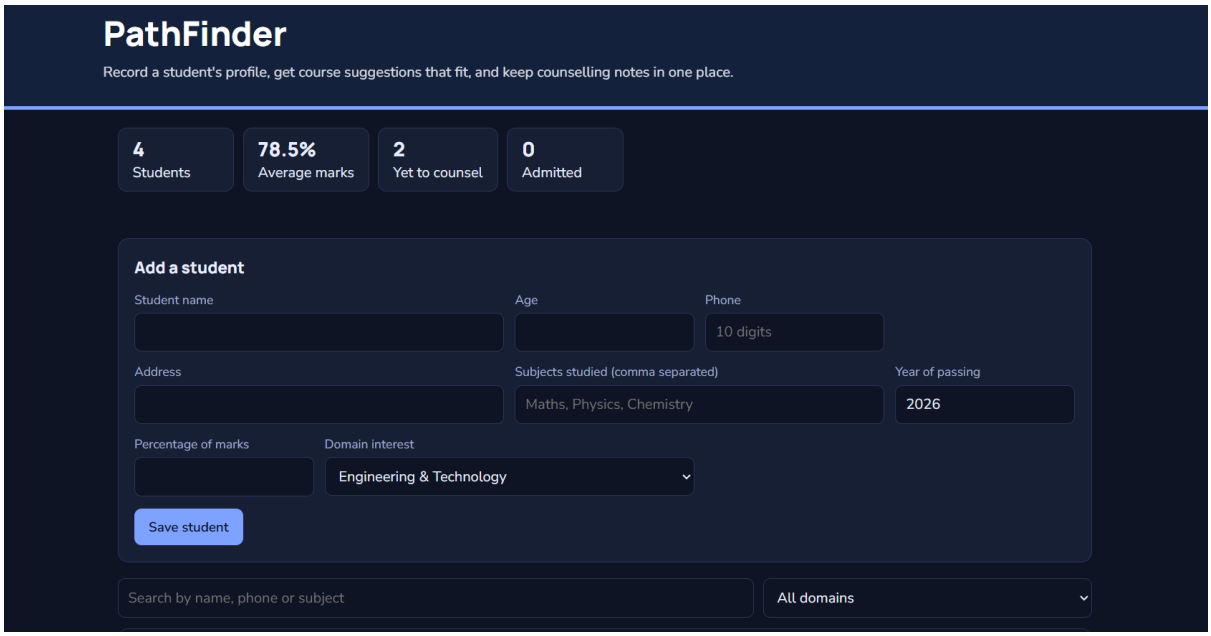
EXPERIMENT 8: Create a simple cloud web application for student counselling system using any Cloud Service Provider to demonstrate SaaS with the fields of student name, age, address, phone, subjects studied, Year of passing, percentage of marks, domain interest, etc.

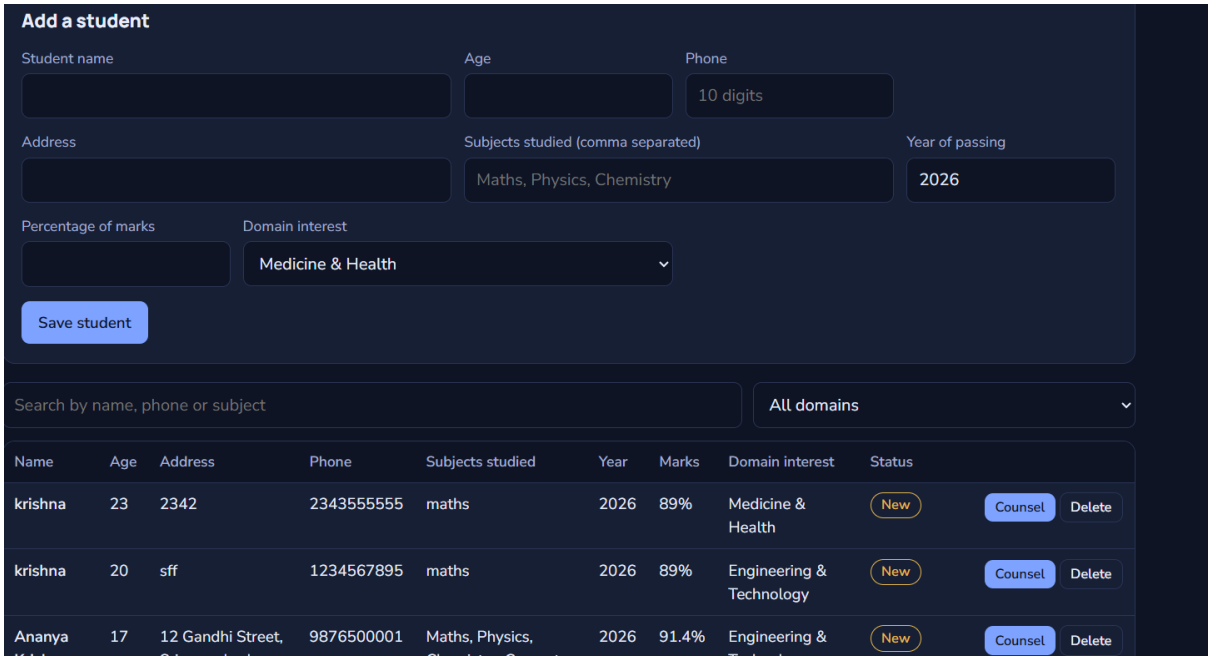
AIM:
To create a simple cloud software application for the Student Counselling System using a Cloud Service Provider and demonstrate Software as a Service (SaaS).

PROCEDURE:
1. Sign in to the chosen Cloud Service Provider (for example Google Cloud, Zoho Creator or Microsoft Azure) and create a new application project.
2. Design the input form and add the required fields: Student Name, Age, Address, Phone, Subjects Studied, Year of Passing, Percentage of Marks and Domain Interest.
3. Create the database table or data store and link each form field to a column.
4. Write the application logic to validate the input, process the data and store the records.
5. Publish the application, open its public URL in a browser, enter sample data and verify the output.

PSEUDOCODE / ALGORITHM:
```
START
  DECLARE name, age, address, phone, subjects, yearOfPassing, percentage, interest, suggestion
  INPUT name, age, address, phone, subjects, yearOfPassing, percentage, interest
  IF name = "" OR phone = "" OR percentage = "" THEN
    PRINT "Required field missing"
    GOTO input form
  END IF
  IF percentage >= 85 AND interest = "Computing" THEN
    suggestion = "B.E. CSE / AI & Data Science"
  ELSE IF percentage >= 70 AND interest = "Computing" THEN
    suggestion = "B.Sc. Computer Science / BCA"
  ELSE IF interest = "Medical" AND percentage >= 75 THEN
    suggestion = "MBBS / B.Pharm / Nursing"
  ELSE
    suggestion = "General degree programmes"
  END IF
  INSERT INTO counselling VALUES (name, age, address, phone, subjects, yearOfPassing, percentage, interest, suggestion)
  PRINT name, suggestion
END
```

OUTPUT:





RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 9: Create a simple cloud software application for Hospital information system for SIMATS Hospital using any Cloud Service Provider to demonstrate SaaS with fields of patient name, age, address, phone, email, Attender name, attender phone, doctor name, diseases, etc.

AIM:

To create a simple cloud software application for the Hospital Information System of SIMATS Hospital using a Cloud Service Provider and demonstrate Software as a Service (SaaS).

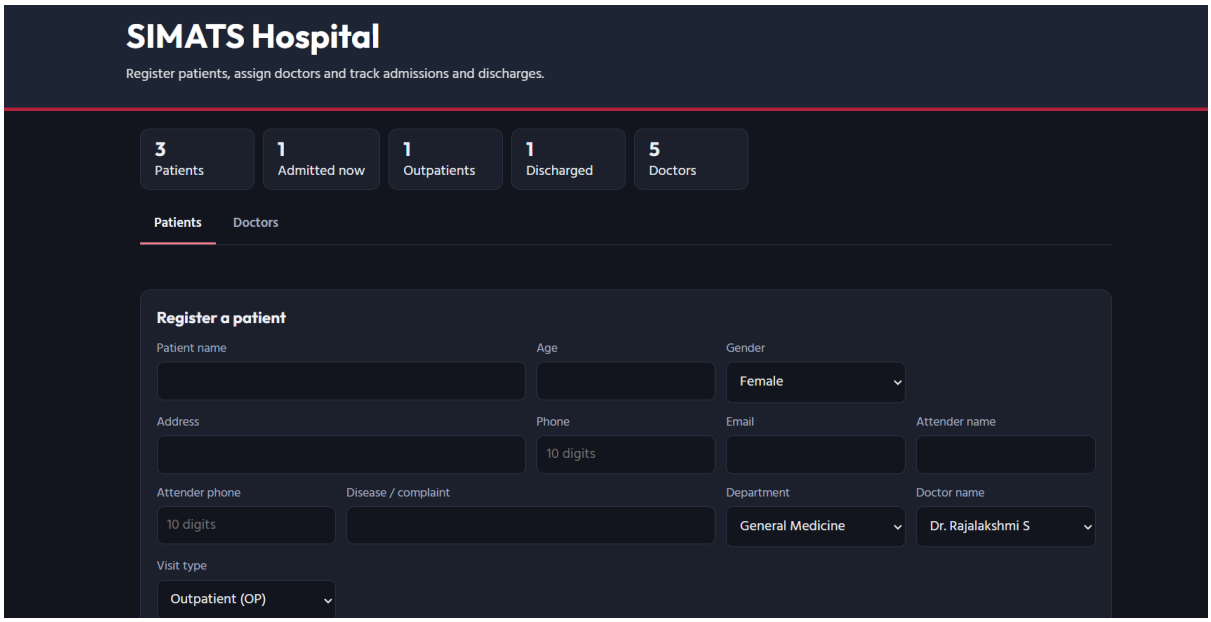
PROCEDURE:

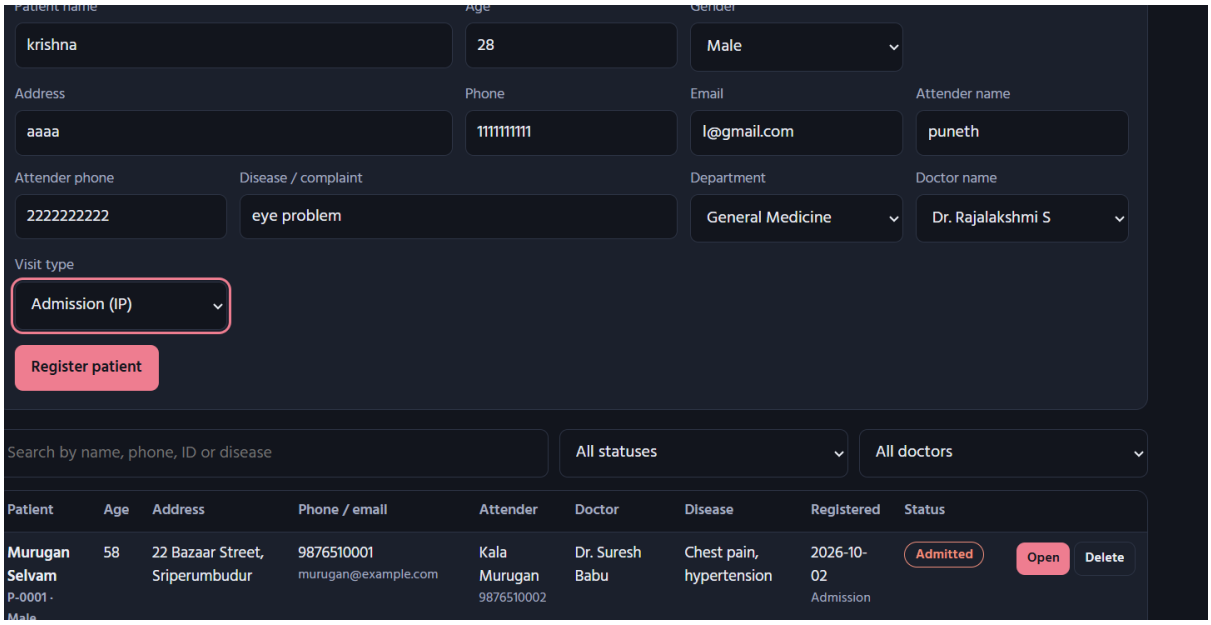
1. Sign in to the chosen Cloud Service Provider (for example Google Cloud, Zoho Creator or Microsoft Azure) and create a new application project.
2. Design the input form and add the required fields: Patient Name, Age, Address, Phone, Email, Attender Name, Attender Phone, Doctor Name and Disease.
3. Create the database table or data store and link each form field to a column.
4. Write the application logic to validate the input, process the data and store the records.
5. Publish the application, open its public URL in a browser, enter sample data and verify the output.

PSEUDOCODE / ALGORITHM:

```
START
  DECLARE patientName, age, address, phone, email, attenderName, attenderPhone, doctor, disease, patientID
  INPUT patientName, age, address, phone, email, attenderName, attenderPhone, disease
  IF patientName = "" OR phone = "" OR disease = "" THEN
    PRINT "Required field missing"
    GOTO input form
  END IF
  patientID = "SIMATS" + NEXT_COUNTER()
  doctor = SELECT doctorName FROM doctors WHERE department = DEPARTMENT_OF(disease)
  INSERT INTO patients VALUES (patientID, patientName, age, address, phone, email, attenderName, attenderPhone, doctor, disease)
  PRINT "Registration Slip", patientID, patientName, doctor
END
```

OUTPUT:





RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 10: Create a simple cloud software application to run a c program to display the student information using any Cloud Service Provider to demonstrate SaaS with the template of student name, reg no, address, phone, age, courses, grades and attendance report, progress report, Semester mark sheet forms.

AIM:

To create a simple cloud software application for the display of student information by running a C program in the cloud using a Cloud Service Provider and demonstrate Software as a Service (SaaS).

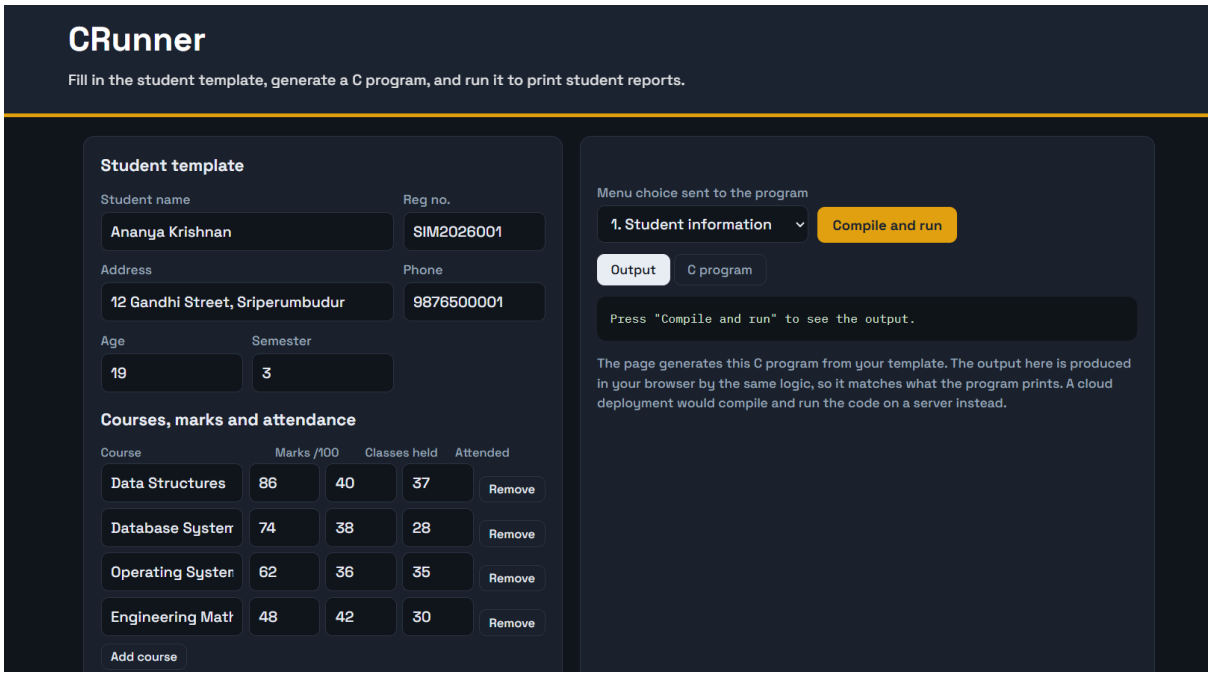
PROCEDURE:

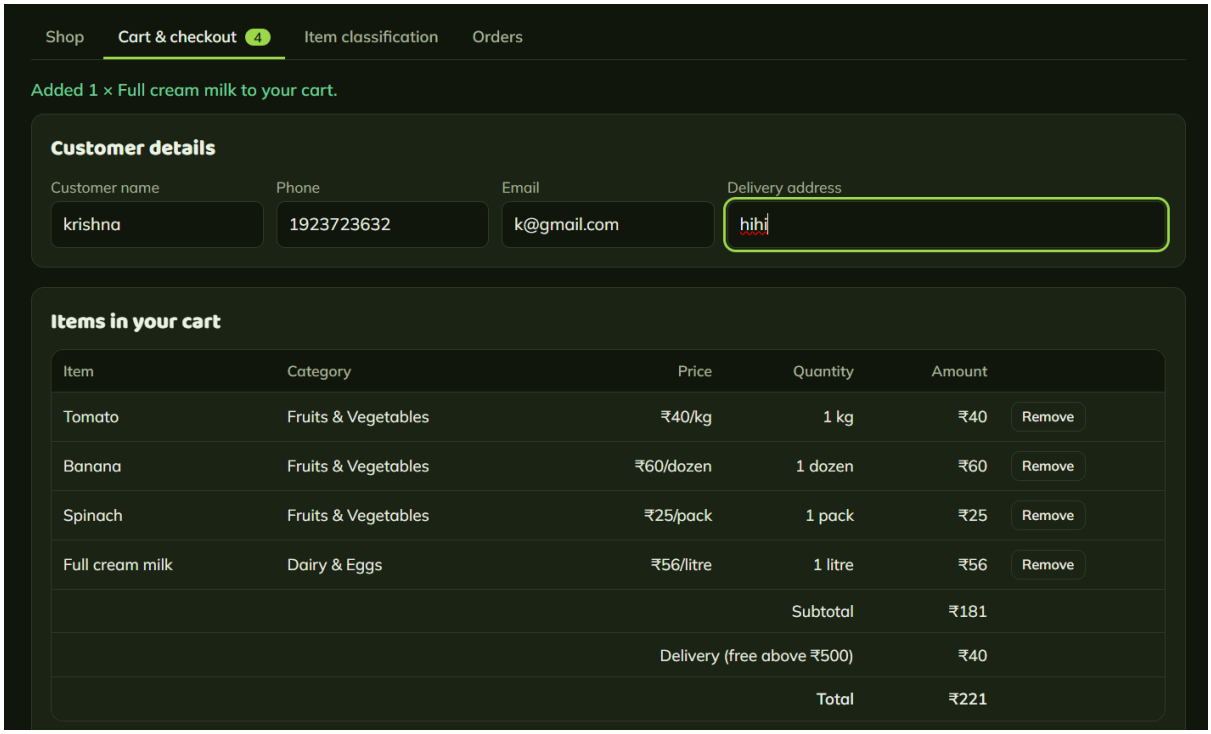
1. Sign in to the chosen Cloud Service Provider (for example Google Cloud, Zoho Creator or Microsoft Azure) and create a new application project.
2. Design the input form and add the required fields: Student Name, Register Number, Address, Phone, Age, Courses, Grades, Attendance Report, Progress Report and Semester Mark Sheet.
3. Create the database table or data store and link each form field to a column.
4. Write the application logic to validate the input, process the data and store the records.
5. Publish the application, open its public URL in a browser, enter sample data and verify the output.

PSEUDOCODE / ALGORITHM:

```
START
  DECLARE STRUCT Student { name, regNo, address, phone, age, course, grade,
attendance, progress, semMarks }
  INPUT name, regNo, address, phone, age, course, grade, attendance, progress, semMarks
  IF name = "" OR regNo = "" THEN
    PRINT "Required field missing"
    GOTO input form
  END IF
  SEND Student TO cloud C compiler
  COMPILE student.c
  EXECUTE student.out
  PRINT name, regNo, address, phone, age, course, grade
  PRINT "Attendance Report :", attendance
  PRINT "Progress Report   :", progress
  PRINT "Semester Marks    :", semMarks
  INSERT INTO students VALUES (name, regNo, address, phone, age, course, grade)
END
```

OUTPUT:





RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 11: Create a simple cloud software application for online super market using any Cloud Service Provider to demonstrate SaaS with the template of customer name, address, phone, email, items, quantity etc. with the item classification form.

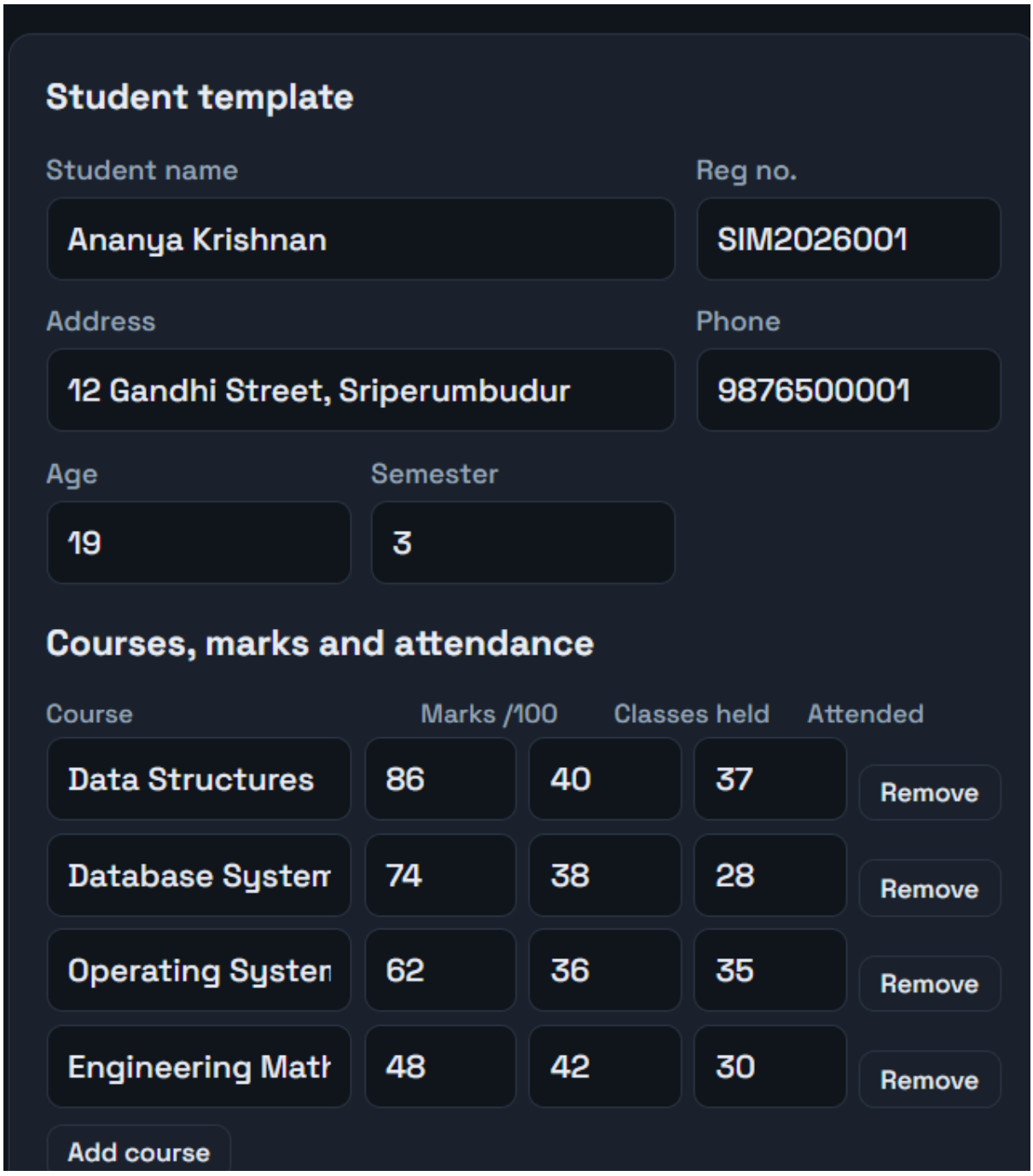
AIM:
To create a simple cloud software application for the Online Super Market using a Cloud Service Provider and demonstrate Software as a Service (SaaS).

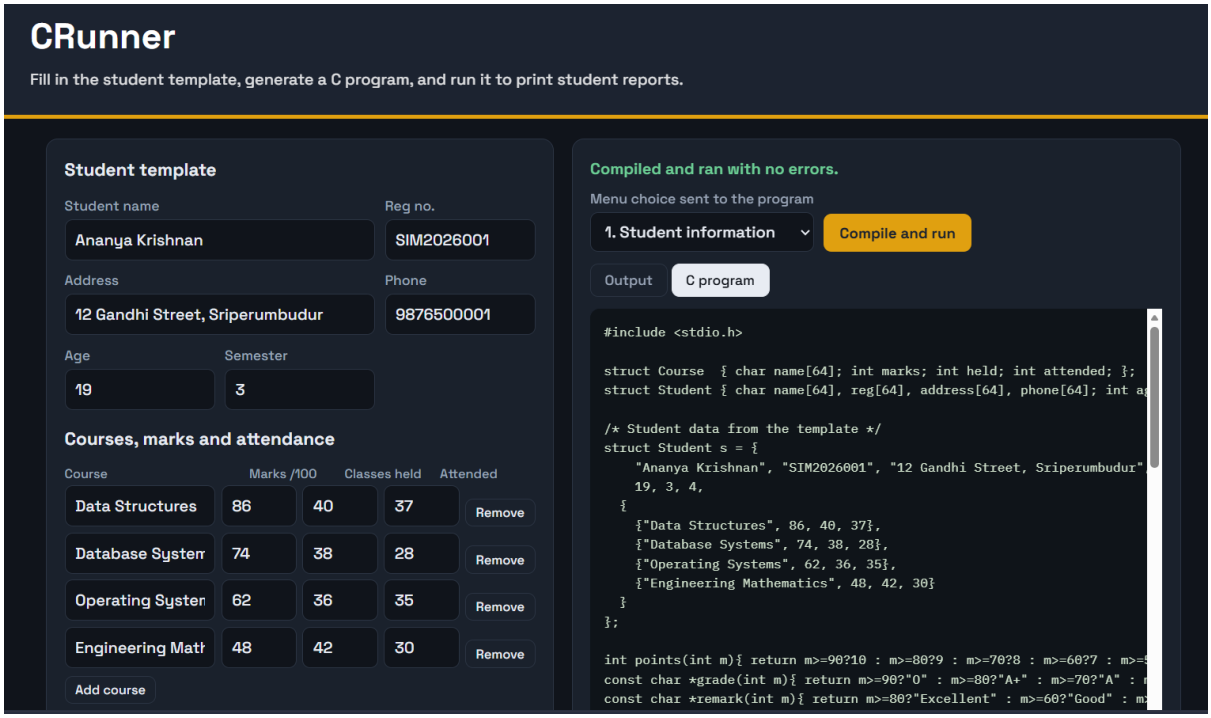
PROCEDURE:
1. Sign in to the chosen Cloud Service Provider (for example Google Cloud, Zoho Creator or Microsoft Azure) and create a new application project.
2. Design the input form and add the required fields: Customer Name, Address, Phone, Email, Items, Quantity and Item Classification (Grocery / Vegetables / Dairy / Household).
3. Create the database table or data store and link each form field to a column.
4. Write the application logic to validate the input, process the data and store the records.
5. Publish the application, open its public URL in a browser, enter sample data and verify the output.

PSEUDOCODE / ALGORITHM:

```
START
  DECLARE customerName, address, phone, email, item[], qty[], category[], price[], bill
  INPUT customerName, address, phone, email, n
  bill = 0
  FOR i = 1 TO n
    INPUT item[i], qty[i]
    category[i] = CLASSIFY(item[i])   // Grocery / Vegetables / Dairy / Household
    price[i] = SELECT price FROM items WHERE name = item[i]
    bill = bill + price[i] * qty[i]
  END FOR
  INSERT INTO orders VALUES (customerName, address, phone, email, bill)
  PRINT "Invoice", customerName
  FOR i = 1 TO n
    PRINT item[i], category[i], qty[i], price[i] * qty[i]
  END FOR
  PRINT "Total Bill =", bill
END
```

OUTPUT:


Student template
Student name
Ananya Krishnan
Reg no.
SIM2026001
Address
12 Gandhi Street, Sriperumbudur
Phone
9876500001
Age
19
Semester
3
Courses, marks and attendance
Course
Marks /100
Classes held
Attended
Data Structures
86
40
37
Remove
Database Systen
74
38
28
Remove
Operating Systen
62
36
35
Remove
Engineering Matl
48
42
30
Remove
Add course



RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 12: Demonstrate virtualization by Installing Type-2 Hypervisor in your device, create and configure VM image with a Host Operating system (Either Windows/Linux).

AIM:
To demonstrate virtualization by installing a Type-2 hypervisor on the host device and creating a configured virtual machine.

PROCEDURE:
1. Enable virtualization (VT-x / AMD-V) in the BIOS and download VirtualBox or VMware Workstation.
2. Install the hypervisor on the host operating system by following the installation wizard.
3. Create a new virtual machine and select the guest OS type (Windows or Linux).
4. Allocate CPU, memory and virtual disk, and attach the OS ISO image.
5. Start the virtual machine, install the guest OS and verify that it boots.

PSEUDOCODE / ALGORITHM:
```
START
  IF BIOS.virtualization (VT-x / AMD-V) = DISABLED THEN
    ENABLE BIOS.virtualization
  END IF
  INSTALL VirtualBox ON hostOS
  vm = CREATE_VM(name = "VM1", os = "Ubuntu 64-bit", cpu = 2, ram = 2048, disk =
20GB)
  ATTACH ISO "ubuntu.iso" TO vm.optical_drive
  POWER_ON vm
  INSTALL guest OS
  IF vm.status = "Running" THEN
    PRINT "Guest OS is running"
  END IF
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 13: Create a Virtual Machine with 1 CPU, 2GB RAM and 15GB storage disk using a Type 2 Virtualization Software.

AIM:
To create a virtual machine with 1 CPU, 2 GB RAM and 15 GB storage using Type-2 virtualization software.

PROCEDURE:
1. Open VirtualBox / VMware Workstation and click New Virtual Machine.

2. Enter the VM name and select the OS type and version.
3. Set the memory size to 2048 MB and the processor count to 1.
4. Create a virtual hard disk of size 15 GB.
5. Attach the ISO, start the VM and check the configuration inside the guest OS.

PSEUDOCODE / ALGORITHM:

```
START
  vm = CREATE_VM(name = "VM1", os = "Ubuntu 64-bit")
  vm.cpu = 1
  vm.ram = 2048          // MB (2 GB)
  vm.disk = CREATE_DISK(size = 15GB, type = "VDI")
  ATTACH ISO TO vm
  POWER_ON vm
  PRINT vm.cpu, vm.ram, vm.disk.size
  IF vm.cpu = 1 AND vm.ram = 2048 AND vm.disk.size = 15GB THEN
    PRINT "Configuration verified"
  END IF
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 14: Create a Virtual Hard Disk and allocate the storage using VM ware Workstation.

AIM:
To create a virtual hard disk and allocate storage to a virtual machine using VMware Workstation.

PROCEDURE:
1. Open VMware Workstation and select the virtual machine.
2. Go to VM Settings and click Add, then choose Hard Disk.
3. Select the disk type (SCSI / SATA) and choose Create a new virtual disk.
4. Specify the disk capacity and choose to store it as a single file or split files.
5. Finish, power on the VM and initialise / format the new disk in the guest OS.

PSEUDOCODE / ALGORITHM:

```
START
  vm = SELECT_VM("VM1")
  disk = NEW_DISK()
  disk.type = "SCSI"
  disk.capacity = 20GB
  disk.file = "D:\VMs\VM1\extra.vmdk"
  disk.splitFiles = FALSE
```

```
  vm.ADD_DEVICE(disk)
  POWER_ON vm
  FORMAT disk IN guest OS
  MOUNT disk
  PRINT "Virtual hard disk added"
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 15: Create a Snapshot of a VM and Test it by loading the Previous Version/Cloned VM.

AIM:
To create a snapshot of a virtual machine and test it by restoring the previous state.

PROCEDURE:
1. Start the virtual machine and create a test file in the guest OS.
2. Take a snapshot of the VM and give it a name.
3. Modify the VM state by deleting the file or installing software.
4. Open the Snapshot Manager and restore the earlier snapshot.
5. Power on the VM and verify that the previous state is restored.

PSEUDOCODE / ALGORITHM:
```
START
  POWER_ON vm
  CREATE_FILE "test.txt" IN vm
  snapshot S1 = TAKE_SNAPSHOT(vm, "S1")
  DELETE_FILE "test.txt" FROM vm
  RESTORE_SNAPSHOT(vm, S1)
  POWER_ON vm
  IF FILE_EXISTS("test.txt", vm) THEN
    PRINT "Snapshot restore successful"
  ELSE
    PRINT "Snapshot restore failed"
  END IF
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 16: Create a Cloning of a VM and Test it by loading the Previous Version/Cloned VM.

AIM:
To create a clone of a virtual machine and test it by loading the cloned VM.

PROCEDURE:
1. Shut down the source virtual machine.
2. Right-click the VM and select Clone.
3. Choose a full clone or linked clone and enter the clone name.
4. Wait for the cloning process to finish.
5. Power on the cloned VM and verify that it has the same data and settings as the source.

PSEUDOCODE / ALGORITHM:

```
START
  POWER_OFF sourceVM
  cloneType = "Full"        // or "Linked"
  cloneVM = CLONE(sourceVM, cloneType, name = "VM1-Clone")
  POWER_ON cloneVM
  IF cloneVM.data = sourceVM.data AND cloneVM.settings = sourceVM.settings THEN
    PRINT "Clone verified"
  ELSE
    PRINT "Clone mismatch"
  END IF
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 17: Change Hardware compatibility of a VM (Either by clone/create new one) which is already created and configured and run simple unix commands.

AIM:
To change the hardware compatibility version of an existing VM and run simple Unix commands in it.

PROCEDURE:
1. Shut down the existing configured virtual machine.
2. Use the Change Hardware Compatibility (or clone) option in VMware Workstation.
3. Select the target hardware version and complete the wizard.

4. Power on the new VM and log in to the Linux guest.
5. Run simple commands such as pwd, ls, date, whoami, uname -a and cal.

PSEUDOCODE / ALGORITHM:

```
START
  POWER_OFF vm
  newVM = CHANGE_HARDWARE_COMPATIBILITY(vm, version = "Workstation
16.x")
  POWER_ON newVM
  OPEN terminal
  EXECUTE pwd
  EXECUTE ls
  EXECUTE date
  EXECUTE whoami
  EXECUTE uname -a
  EXECUTE cal
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 18: Install Virtualbox/VMware Workstation with different flavours of linux or windows OS on top of your present windows environment.

AIM:
To install VirtualBox / VMware Workstation on Windows and run different flavours of Linux or Windows as guest operating systems.

PROCEDURE:
1. Download the VirtualBox / VMware Workstation installer and run it as administrator.
2. Complete the installation and restart the system if required.
3. Create a new virtual machine for the first guest OS (for example Ubuntu).
4. Create another virtual machine for a second guest OS (for example Windows or Fedora).
5. Start each virtual machine and verify that the guest operating systems work.

PSEUDOCODE / ALGORITHM:

```
START
  INSTALL hypervisor ON Windows
  guestList = {"Ubuntu", "Fedora", "Windows"}
  FOR EACH guest IN guestList
    vm = CREATE_VM(name = guest, os = guest)
    ATTACH ISO(guest) TO vm
    POWER_ON vm
    INSTALL guest OS
```

```
    PRINT guest, "is running"
  END FOR
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 19: Install a C compiler in the virtual machine created using virtual box / VM Ware Workstation / Player and execute Simple C Programs.

AIM:
To install a C compiler in a virtual machine and execute simple C programs.

PROCEDURE:
1. Start the Linux virtual machine and open the terminal.
2. Update the package list and install the compiler using sudo apt install gcc.
3. Verify the installation with gcc --version.
4. Write a simple C program (for example Hello World) using a text editor.
5. Compile with gcc hello.c -o hello and run it with ./hello.

PSEUDOCODE / ALGORITHM:
```
START
  EXECUTE sudo apt update
  EXECUTE sudo apt install gcc -y
  EXECUTE gcc --version
  CREATE FILE hello.c
    #include <stdio.h>
    int main() {
      printf("Hello World");
      return 0;
    }
  EXECUTE gcc hello.c -o hello
  EXECUTE ./hello
  PRINT "Hello World"
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 20: Install Virtualbox with different flavours of linux or windows OS on top of windows 10 using custom installation.

AIM:
To install VirtualBox on Windows 10 using custom installation and run different guest operating systems.

PROCEDURE:
1. Download the VirtualBox installer for Windows 10 and start the setup.
2. Choose Custom Setup and select the required features and installation folder.
3. Complete the installation and the network driver installation prompts.
4. Create virtual machines for Linux and Windows guest operating systems.
5. Boot each VM and verify the installation.

PSEUDOCODE / ALGORITHM:
```
START
  RUN VirtualBox-installer.exe
  SELECT "Custom Setup"
  features = {"VirtualBox Application", "USB Support", "Networking"}
  SET installPath = "C:\Program Files\Oracle\VirtualBox"
  INSTALL features AT installPath
  LAUNCH VirtualBox
  FOR EACH guest IN {"Linux", "Windows"}
    vm = CREATE_VM(guest)
    POWER_ON vm
    VERIFY vm.status = "Running"
  END FOR
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 21: Install Virtualbox/VMware Workstation with different flavours of linux or windows OS on top of windows 7 or 8.

AIM:
To install VirtualBox / VMware Workstation on Windows 7 or 8 and run different guest operating systems.

PROCEDURE:
1. Download a version of the hypervisor that supports Windows 7 or 8.
2. Run the installer and complete the installation wizard.

3. Create a new virtual machine and select the guest OS type.
4. Attach the ISO image and install the guest OS.
5. Start the VM and confirm that it runs correctly.

PSEUDOCODE / ALGORITHM:

```
START
  IF hostOS = "Windows 7" OR hostOS = "Windows 8" THEN
    installer = DOWNLOAD(compatible hypervisor version)
  END IF
  RUN installer
  vm = CREATE_VM(name, guestOS)
  ATTACH ISO TO vm
  POWER_ON vm
  INSTALL guestOS
  IF vm.status = "Running" THEN
    PRINT "Guest OS working"
  END IF
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 22: To write a procedure to run the virtual machine of different configuration and to check how many virtual machines can be utilized at a particular time.

AIM:
To run virtual machines of different configurations and find how many can run at the same time on the host.

PROCEDURE:
1. Note the host CPU cores and RAM using Task Manager.
2. Create virtual machines with different CPU, RAM and disk configurations.
3. Start the virtual machines one by one.
4. Monitor the host CPU and memory usage after each VM is started.
5. Record the maximum number of VMs that run smoothly at the same time.

PSEUDOCODE / ALGORITHM:

```
START
  hostCPU = GET_HOST_CPU_CORES()
  hostRAM = GET_HOST_RAM()
  count = 0
  usedRAM = 0
  WHILE usedRAM + vmRAM[count + 1] <= hostRAM AND cpuUsage < 90
```

```
    START vm[count + 1]
    usedRAM = usedRAM + vmRAM[count + 1]
    count = count + 1
    cpuUsage = GET_CPU_USAGE()
  END WHILE
  PRINT "Maximum VMs running =", count
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 23: Write a Procedure To Attach Virtual Block To The Virtual Machine And Check Whether It Holds the Data Even After The Release Of The Virtual Machine.

AIM:
To attach a virtual block storage device to a virtual machine and verify that the data persists after the VM is released.

PROCEDURE:
1. Create a new virtual disk and attach it to the running virtual machine.
2. Partition, format and mount the disk inside the guest OS.
3. Create a file with sample data on the mounted disk.
4. Unmount and detach the disk, then shut down or delete the VM.
5. Attach the same disk to another VM, mount it and check that the data is present.

PSEUDOCODE / ALGORITHM:
```
START
  D1 = CREATE_DISK(size = 5GB)
  ATTACH D1 TO VM1
  PARTITION D1
  FORMAT D1 AS ext4
  MOUNT D1 AT /mnt/data
  WRITE "sample data" TO /mnt/data/data.txt
  UNMOUNT D1
  DETACH D1 FROM VM1
  DELETE VM1
  ATTACH D1 TO VM2
  MOUNT D1 AT /mnt/data
  IF FILE_EXISTS("/mnt/data/data.txt") THEN
    PRINT "Data is persistent"
  ELSE
    PRINT "Data lost"
  END IF
```

END

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 24: To showcase the virtual machine migration based on the certain condition from one host to the other.

AIM:
To demonstrate virtual machine migration from one host to another based on a given condition.

PROCEDURE:
1. Set up two hosts with the hypervisor and shared storage or a network connection.
2. Run a virtual machine on the source host.
3. Define the migration condition, for example high CPU load on the source host.
4. Migrate the virtual machine to the destination host (live migration or export / import).
5. Verify that the VM is running on the destination host with its data intact.

PSEUDOCODE / ALGORITHM:
```
START
  threshold = 80
  WHILE TRUE
    load = GET_CPU_LOAD(sourceHost)
    IF load > threshold THEN
      destHost = SELECT host WITH lowest load
      MIGRATE(vm, sourceHost, destHost)
      RESUME vm ON destHost
      BREAK
    END IF
  END WHILE
  IF vm.host = destHost AND vm.status = "Running" THEN
    PRINT "Migration successful"
  END IF
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 25: Demonstrate Infrastructure as a Service (IaaS) by creating a resources group using a Public Cloud Service Provider (Azure), configure with minimum CPU, RAM, and Storage.

AIM:
To demonstrate IaaS by creating a resource group in Microsoft Azure and configuring minimum CPU, RAM and storage.

PROCEDURE:
1. Sign in to the Azure portal.
2. Search for Resource groups and click Create.
3. Select the subscription, enter the resource group name and choose the region.
4. Review and create the resource group.
5. Add resources inside it with minimum size (for example B1s VM with 1 vCPU, 1 GB RAM and a standard disk).

PSEUDOCODE / ALGORITHM:
START
  LOGIN TO Azure portal
  az group create --name MyResourceGroup --location eastus
  az vm create --resource-group MyResourceGroup --name MyVM --image Ubuntu2204 --size Standard_B1s --storage-sku Standard_LRS
  az resource list --resource-group MyResourceGroup
  PRINT "Resource group and resources created"
END

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 26: Demonstrate Infrastructure as a Service (IaaS) by creating a Virtual Machine using a Public Cloud Service Provider (Azure), configure with required memory and CPU.

AIM:
To demonstrate IaaS by creating a virtual machine in Azure with the required memory and CPU.

PROCEDURE:
1. Sign in to the Azure portal and click Create a resource, then Virtual Machine.
2. Select the resource group, VM name, region and the OS image.
3. Choose the VM size according to the required CPU and memory.
4. Set the administrator username and password and allow the RDP / SSH port.

5. Review and create the VM and wait for the deployment to complete.

PSEUDOCODE / ALGORITHM:

```
START
  LOGIN TO Azure portal
  resourceGroup = "MyResourceGroup"
  vmName = "MyVM"
  region = "eastus"
  image = "Win2019Datacenter"
  size = "Standard_B2s"      // 2 vCPU, 4 GB RAM
  az vm create --resource-group resourceGroup --name vmName --image image --size size --
admin-username azureuser --admin-password ********
  az vm open-port --port 3389 --resource-group resourceGroup --name vmName
  status = az vm get-instance-view --name vmName
  IF status = "Running" THEN
    PRINT "VM deployed"
  END IF
END
```

OUTPUT:

[Insert Output Screenshot / Log Here]

RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 27: Demonstrate Infrastructure as a Service (IaaS) by establishing the remote connection, launch the created VM image and run in your desktop.

AIM:

To connect remotely to the Azure virtual machine and run it from the local desktop.

PROCEDURE:

1. Open the created VM in the Azure portal and copy its public IP address.
2. Download the RDP file (Windows VM) or note the SSH command (Linux VM).
3. Open Remote Desktop Connection or a terminal on the local computer.
4. Enter the IP address, username and password to connect.
5. Verify that the VM desktop / shell opens and run a few commands or applications.

PSEUDOCODE / ALGORITHM:

```
START
  ip = az vm show -d --name MyVM --query publicIps
  IF vmType = "Windows" THEN
    OPEN Remote Desktop Connection
    CONNECT ip:3389
  ELSE
    EXECUTE ssh azureuser@ip
  END IF
```

```
  INPUT username, password
  IF authentication = SUCCESS THEN
    OPEN remote session
    RUN commands / applications
  ELSE
    PRINT "Login failed"
  END IF
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 28: Demonstrate Platform as a Service (PaaS) create and configure a new VM Image in any Public Cloud Service Provider.

AIM:
To demonstrate PaaS by creating and configuring a new VM image in a public cloud.

PROCEDURE:
1. Sign in to the public cloud portal (for example Azure).
2. Select Create a resource and choose the required platform image.
3. Configure the region, size, disk and networking.
4. Review the settings and create the virtual machine.
5. Open the deployed resource and verify that it is running.

PSEUDOCODE / ALGORITHM:
```
START
  LOGIN TO Azure portal
  image = SELECT FROM marketplace WHERE name = "Ubuntu Server"
  az vm create --resource-group MyRG --name MyVM --image image --size Standard_B1s --
location eastus
  status = az vm get-instance-view --name MyVM
  IF status = "Running" THEN
    PRINT "VM image deployed"
  END IF
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 29: Create a Simple Web Application using Java or Python and host it in any Public Cloud Service Provider (Azure/GCP/AWS) to demonstrate Platform as a Service (PaaS).

AIM:
To create a simple web application in Python or Java and host it on a public cloud PaaS.

PROCEDURE:
1. Write a simple web application (for example a Flask app that returns a welcome page).
2. Test the application on the local system.
3. Create an App Service / App Engine / Elastic Beanstalk instance in the cloud portal.
4. Deploy the application code to the service using the portal, Git or CLI.
5. Open the public URL and verify that the web page is displayed.

PSEUDOCODE / ALGORITHM:
```
START
  CREATE FILE app.py
    from flask import Flask
    app = Flask(__name__)
    @app.route("/")
    def home():
      return "Welcome to PaaS Demo"
    if __name__ == "__main__":
      app.run()
  EXECUTE python app.py            // test locally
  az webapp up --name MyWebApp --resource-group MyRG --runtime "PYTHON:3.10"
  OPEN https://MyWebApp.azurewebsites.net
  PRINT "Welcome to PaaS Demo"
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 30: Demonstrate Storage as a Service (SaaS) create and configure a new VM Image in any Public Cloud Service Provider.

AIM:
To demonstrate Storage as a Service by creating and configuring a new VM image with attached storage in a public cloud.

PROCEDURE:
1. Sign in to the cloud portal and start creating a new virtual machine.
2. Select the VM image and configure the size and region.
3. Add a managed data disk or storage account to the VM.
4. Review and create the VM.
5. Connect to the VM and verify that the storage is available.

PSEUDOCODE / ALGORITHM:
```
START
  LOGIN TO Azure portal
  az vm create --resource-group MyRG --name MyVM --image Ubuntu2204 --size
Standard_B1s
  az vm disk attach --resource-group MyRG --vm-name MyVM --name DataDisk --new --
size-gb 32
  CONNECT TO MyVM
  EXECUTE lsblk
  FORMAT /dev/sdc AS ext4
  MOUNT /dev/sdc AT /datadrive
  IF DIRECTORY_EXISTS("/datadrive") THEN
    PRINT "Storage is available"
  END IF
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 31: Create a Storage service using any Public Cloud Service Provider (Azure/GCP/AWS) and check the public accessibility of the stored file to demonstrate Storage as a Service.

AIM:
To create a cloud storage service, upload a file and check its public accessibility.

PROCEDURE:
1. Create a storage account / bucket in the cloud portal.
2. Create a container and upload a sample file.
3. Set the access level of the container to public (blob / read access).
4. Copy the URL of the uploaded file.
5. Open the URL in a browser and verify that the file is accessible.

PSEUDOCODE / ALGORITHM:
```
START
  az storage account create --name mystorage123 --resource-group MyRG --sku
Standard_LRS
```

```
  az storage container create --name mycontainer --account-name mystorage123 --public-
access blob
  az storage blob upload --container-name mycontainer --file sample.txt --name sample.txt --
account-name mystorage123
  url = "https://mystorage123.blob.core.windows.net/mycontainer/sample.txt"
  response = HTTP_GET(url)
  IF response.status = 200 THEN
    PRINT "File is publicly accessible"
  ELSE
    PRINT "Access denied"
  END IF
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 32: Database as a Service (DaaS) create and configure a new VM Image in any Public Cloud Service Provider.

AIM:
To demonstrate Database as a Service by creating and configuring a new VM image for a database in a public cloud.

PROCEDURE:
1. Sign in to the cloud portal and select Create a resource.
2. Choose a VM image that contains a database (for example SQL Server or MySQL).
3. Configure the size, region, disk and inbound port for the database.
4. Review and create the virtual machine.
5. Connect to the VM and verify that the database service is running.

PSEUDOCODE / ALGORITHM:
```
START
  LOGIN TO Azure portal
  image = "SQL Server 2019 on Windows Server 2019"
  az vm create --resource-group MyRG --name DBVM --image image --size
Standard_D2s_v3
  az vm open-port --port 1433 --resource-group MyRG --name DBVM
  CONNECT TO DBVM
  status = CHECK_SERVICE("MSSQLSERVER")
  IF status = "Running" THEN
    PRINT "Database service is running"
  END IF
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 33: Create a SQL storage service and perform a basic query using any Public Cloud Service Provider (Azure/GCP/AWS) to demonstrate Database as a Service (DaaS).

AIM:
To create a cloud SQL database service and run a basic query on it.

PROCEDURE:
1. Create a SQL server and a SQL database in the cloud portal.
2. Configure the firewall to allow the client IP address.
3. Open the Query Editor or connect using a SQL client.
4. Create a table and insert sample records.
5. Run a SELECT query and check the result.

PSEUDOCODE / ALGORITHM:
START
  az sql server create --name mysqlserver --resource-group MyRG --admin-user sqladmin --admin-password ********
  az sql db create --name StudentDB --server mysqlserver --resource-group MyRG
  az sql server firewall-rule create --server mysqlserver --name AllowClient --start-ip-address clientIP --end-ip-address clientIP
  CONNECT TO StudentDB
  CREATE TABLE Student (id INT, name VARCHAR(50), marks INT)
  INSERT INTO Student VALUES (1, 'Arun', 85)
  INSERT INTO Student VALUES (2, 'Priya', 90)
  SELECT * FROM Student
  PRINT result rows
END

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 34: Demonstrate Platform as a Service (PaaS) create and configure a new VM Image in any Public Cloud Service Provider to get the Ubuntu Server 22.04 LTS.

AIM:
To create and configure a new VM image of Ubuntu Server 22.04 LTS in a public cloud service provider.

PROCEDURE:
1. Sign in to the public cloud portal (for example Azure) and click Create a resource.
2. Search for and select the image: Ubuntu Server 22.04 LTS.
3. Enter the resource group, VM name and region, and choose a suitable VM size.
4. Set the administrator credentials and allow the required inbound port (SSH).
5. Review and create the VM, then connect to it and verify that it runs.

PSEUDOCODE / ALGORITHM:
START
  LOGIN TO Azure portal
  az vm create --resource-group MyRG --name UbuntuVM --image Canonical:0001-com-ubuntu-server-jammy:22_04-lts:latest --size Standard_B1s --admin-username azureuser --generate-ssh-keys
  ip = az vm show -d --name UbuntuVM --query publicIps
  EXECUTE ssh azureuser@ip
  EXECUTE lsb_release -a
  PRINT "Ubuntu Server 22.04 LTS running"
END

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 35: Demonstrate Platform as a Service (PaaS) create and configure a new VM Image in any Public Cloud Service Provider to get windows server Gen2.

AIM:
To create and configure a new VM image of Windows Server Gen2 in a public cloud service provider.

PROCEDURE:
1. Sign in to the public cloud portal (for example Azure) and click Create a resource.
2. Search for and select the image: Windows Server Gen2.
3. Enter the resource group, VM name and region, and choose a suitable VM size.
4. Set the administrator credentials and allow the required inbound port (RDP).
5. Review and create the VM, then connect to it and verify that it runs.

PSEUDOCODE / ALGORITHM:

START
  LOGIN TO Azure portal
  az vm create --resource-group MyRG --name WinVM --image MicrosoftWindowsServer:WindowsServer:2019-datacenter-gensecond:latest --size Standard_B2s --admin-username azureuser --admin-password ********
  az vm open-port --port 3389 --resource-group MyRG --name WinVM
  ip = az vm show -d --name WinVM --query publicIps
  CONNECT RDP ip:3389
  PRINT "Windows Server Gen2 running"
END

OUTPUT:

[Insert Output Screenshot / Log Here]

RESULT:

Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 36: Demonstrate Platform as a Service (PaaS) create and configure a new VM Image in any Public Cloud Service Provider to get Microsoft windows 10.

AIM:

To create and configure a new VM image of Microsoft Windows 10 in a public cloud service provider.

PROCEDURE:

1. Sign in to the public cloud portal (for example Azure) and click Create a resource.
2. Search for and select the image: Microsoft Windows 10.
3. Enter the resource group, VM name and region, and choose a suitable VM size.
4. Set the administrator credentials and allow the required inbound port (RDP).
5. Review and create the VM, then connect to it and verify that it runs.

PSEUDOCODE / ALGORITHM:

START
  LOGIN TO Azure portal
  az vm create --resource-group MyRG --name Win10VM --image MicrosoftWindowsDesktop:Windows-10:win10-22h2-pro:latest --size Standard_B2s --admin-username azureuser --admin-password ********
  az vm open-port --port 3389 --resource-group MyRG --name Win10VM
  ip = az vm show -d --name Win10VM --query publicIps
  CONNECT RDP ip:3389
  PRINT "Windows 10 running"
END

OUTPUT:

[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 37: Demonstrate Platform as a Service (PaaS) create and configure a new VM Image in any Public Cloud Service Provider to get Windows Server 2019 Small.

AIM:
To create and configure a new VM image of Windows Server 2019 (small size) in a public cloud service provider.

PROCEDURE:
1. Sign in to the public cloud portal (for example Azure) and click Create a resource.
2. Search for and select the image: Windows Server 2019 (small size).
3. Enter the resource group, VM name and region, and choose a suitable VM size.
4. Set the administrator credentials and allow the required inbound port (RDP).
5. Review and create the VM, then connect to it and verify that it runs.

PSEUDOCODE / ALGORITHM:
```
START
  LOGIN TO Azure portal
  az vm create --resource-group MyRG --name Win2019VM --image
MicrosoftWindowsServer:WindowsServer:2019-datacenter-smalldisk:latest --size
Standard_B2s --admin-username azureuser --admin-password ********
  az vm open-port --port 3389 --resource-group MyRG --name Win2019VM
  ip = az vm show -d --name Win2019VM --query publicIps
  CONNECT RDP ip:3389
  PRINT "Windows Server 2019 Small running"
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 38: Perform the basic configuration setup for installing HADOOP 2.x like creating the HDUSER and SSH localhost.

AIM:
To perform the basic configuration required for installing Hadoop 2.x, including creating the hduser and configuring SSH for localhost.

PROCEDURE:
1. Install Java (JDK) and check it with java -version.
2. Create a group and user: sudo addgroup hadoop and sudo adduser --ingroup hadoop hduser.
3. Install the SSH server using sudo apt install openssh-server.
4. Switch to hduser and generate an SSH key: ssh-keygen -t rsa -P "".
5. Add the public key to authorized_keys and test with ssh localhost.

PSEUDOCODE / ALGORITHM:
```
START
  EXECUTE sudo apt install default-jdk -y
  EXECUTE java -version
  EXECUTE sudo addgroup hadoop
  EXECUTE sudo adduser --ingroup hadoop hduser
  EXECUTE sudo apt install openssh-server -y
  SWITCH USER hduser
  EXECUTE ssh-keygen -t rsa -P ""
  EXECUTE cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
  EXECUTE ssh localhost
  IF login succeeds WITHOUT password THEN
    PRINT "SSH setup complete"
  END IF
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 39: Install Hadoop 2.x and configure the Name Node and Data Node.

AIM:
To install Hadoop 2.x and configure the NameNode and DataNode.

PROCEDURE:
1. Download the Hadoop 2.x tarball, extract it and move it to /usr/local/hadoop.
2. Set JAVA_HOME and HADOOP_HOME in .bashrc and hadoop-env.sh.
3. Edit core-site.xml (fs.defaultFS) and hdfs-site.xml (replication, namenode and datanode directories).
4. Create the NameNode and DataNode directories and give hduser ownership.
5. Format the NameNode using hdfs namenode -format.

PSEUDOCODE / ALGORITHM:
```
START
  EXECUTE tar -xzf hadoop-2.x.tar.gz
  EXECUTE sudo mv hadoop-2.x /usr/local/hadoop
```

```
  SET JAVA_HOME = /usr/lib/jvm/default-java
  SET HADOOP_HOME = /usr/local/hadoop
  EDIT core-site.xml    SET fs.defaultFS = hdfs://localhost:9000
  EDIT hdfs-site.xml    SET dfs.replication = 1
                    SET dfs.namenode.name.dir = file:/usr/local/hadoop_store/hdfs/namenode
                    SET dfs.datanode.data.dir = file:/usr/local/hadoop_store/hdfs/datanode
  EXECUTE sudo mkdir -p /usr/local/hadoop_store/hdfs/namenode
  EXECUTE sudo mkdir -p /usr/local/hadoop_store/hdfs/datanode
  EXECUTE sudo chown -R hduser:hadoop /usr/local/hadoop_store
  EXECUTE hdfs namenode -format
  PRINT "NameNode formatted successfully"
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.

EXPERIMENT 40: Launch the Hadoop 2.x and test the Map-Reduce Platform with Hadoop.

AIM:
To start the Hadoop 2.x services and test the MapReduce platform using a sample job.

PROCEDURE:
1. Start the HDFS and YARN services using start-dfs.sh and start-yarn.sh.
2. Verify that NameNode, DataNode, ResourceManager and NodeManager are running using jps.
3. Create an HDFS directory and copy an input text file into it.
4. Run the word count job: hadoop jar hadoop-mapreduce-examples.jar wordcount /input /output.
5. Display the result using hdfs dfs -cat /output/part-r-00000.

PSEUDOCODE / ALGORITHM:

```
START
  EXECUTE start-dfs.sh
  EXECUTE start-yarn.sh
  EXECUTE jps
  hdfs dfs -mkdir /input
  hdfs dfs -put input.txt /input
  hadoop jar hadoop-mapreduce-examples.jar wordcount /input /output
  FUNCTION Map(line)
    FOR EACH word IN line
      EMIT (word, 1)
    END FOR
  END FUNCTION
```

```
  FUNCTION Reduce(word, counts[])
    sum = 0
    FOR EACH c IN counts
      sum = sum + c
    END FOR
    EMIT (word, sum)
  END FUNCTION
  hdfs dfs -cat /output/part-r-00000
END
```

OUTPUT:
[Insert Output Screenshot / Log Here]

RESULT:
Thus, the experiment was successfully performed and executed, and the expected output was obtained.